Purple Team Assessment of a Vulnerable Web Application: Attack Simulation, Forensic Analysis, and Defensive Implementation

Opeyemi A. Ajadi

Department of Computer and Mathematical Sciences, Lewis University

[Course Number: Course Name]

[Instructor Name]

October 25, 2026

Abstract

This report documents a purple team security assessment, an exercise that combines offensive testing (the red team) and defensive analysis (the blue team), carried out on a deliberately vulnerable website that the author built and ran in an isolated test lab. The project had two phases. In the first phase the website had no protection and was compromised through a complete, multi stage attack chain: reconnaissance (gathering information about the target), a SQL injection authentication bypass (using crafted input to log in without a valid password), credential brute force (rapidly guessing passwords), a file upload used to plant a web shell (a malicious file placed on the server), operating system command injection leading to remote code execution (running the attacker's own commands on the server), and data exfiltration (copying data out). The incident was then reconstructed using only the website's log files and mapped to the MITRE ATT&CK framework, a widely used catalog of attacker techniques. In the second phase, a web application firewall (ModSecurity with the OWASP Core Rule Set) and a custom Python automated response program were added, and the same attack chain was run again. The firewall blocked the injection attacks at the network edge, and the automated response program defeated the brute force that the firewall could not detect. The results show how preventive controls and detective controls complement one another in protecting a public facing website.

Keywords: web application security, SQL injection, command injection, MITRE ATT&CK, web application firewall, incident response, purple team

Purple Team Assessment of a Vulnerable Web Application

Public facing websites and online services are among the most frequently targeted assets in computing, and the techniques used to compromise them are well documented (OWASP Foundation, 2021). A purple team exercise examines these techniques from both sides, first by performing an attack (the offensive, or red team, role) and then by detecting and stopping that same attack (the defensive, or blue team, role), all against a single controlled target. This report presents a purple team assessment of a website that the author designed, attacked, and then defended in an isolated test lab created only for this research.

The assessment had three objectives: first, to carry out a realistic, multi stage attack chain against the website; second, to act as a security operations center (SOC) analyst and reconstruct the attack using only the website's log files; and third, to deploy protective controls and demonstrate that they stop the same attacks. The work was divided into two phases, which mirror how a real detection program matures, moving from reactive incident response toward proactive prevention.

Laboratory Environment and Methodology

Target Application

The target was a small website built with Flask, a framework for building websites in the Python programming language. It presented a client portal with a login page, an authenticated dashboard (the page shown after signing in), a file upload feature, and a network diagnostics tool that pings another host (checks whether it is reachable). User accounts were stored in a SQLite database, a lightweight database kept in a single file. Six vulnerabilities (security weaknesses) were introduced on purpose to support the attack chain: a login query open to SQL injection; no brute force protection, meaning no limit on password attempts; an unrestricted file upload that

accepted any file type; operating system command injection in the ping tool; plaintext password storage, where passwords are saved as readable text rather than scrambled, or hashed; and the application running as the root user, the account with full control of the system. Verbose logging was enabled so that all activity was recorded for later forensic analysis, the examination of evidence after an incident.

Attacker Platform

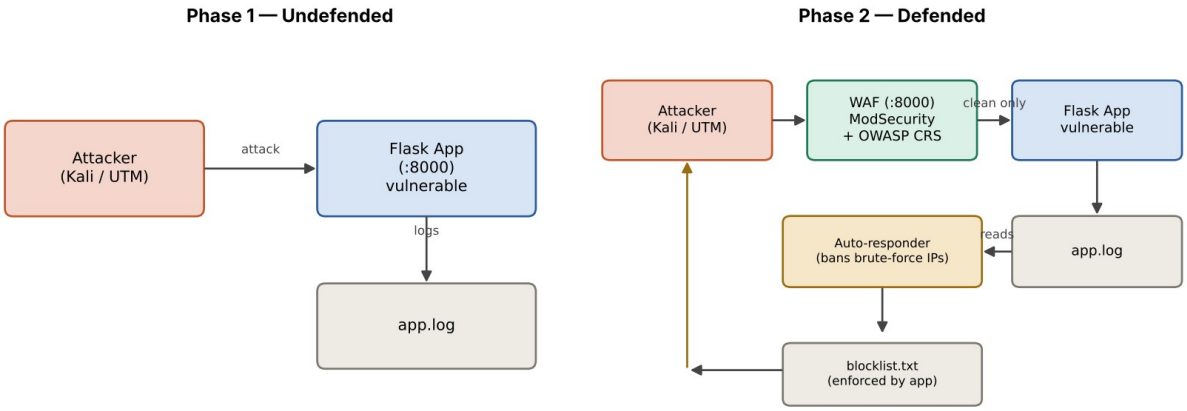
Offensive activity was performed from Kali Linux, a Linux distribution that ships with many security testing tools preinstalled, running as a virtual machine (a computer that runs inside another computer) on an Apple Silicon laptop. The tools used are all standard and freely available, including nmap, gobuster, sqlmap, hydra, and curl. The attacking machine and the target communicated over a private virtual network isolated from the internet and from any real systems.

Defensive Controls (Phase 2)

For the second phase, a web application firewall (WAF), a filter that inspects every incoming request and blocks those that look malicious, was placed in front of the website. The specific product was ModSecurity combined with the OWASP Core Rule Set, a widely used set of detection rules (OWASP Foundation, 2024). The website was reconfigured so that all traffic had to pass through the WAF first. In addition, a custom automated response program was written in Python to monitor the website's log and automatically block any source address showing brute force behavior. The overall architecture is shown in Figure 1.

Figure 1

Laboratory Architecture for Phase 1 (Undefended) and Phase 2 (Defended)



Note. In Phase 2 all traffic is routed through the ModSecurity web application firewall, and a Python automated response program monitors the log and writes offending addresses to a blocklist enforced by the application. Diagram created by the author.

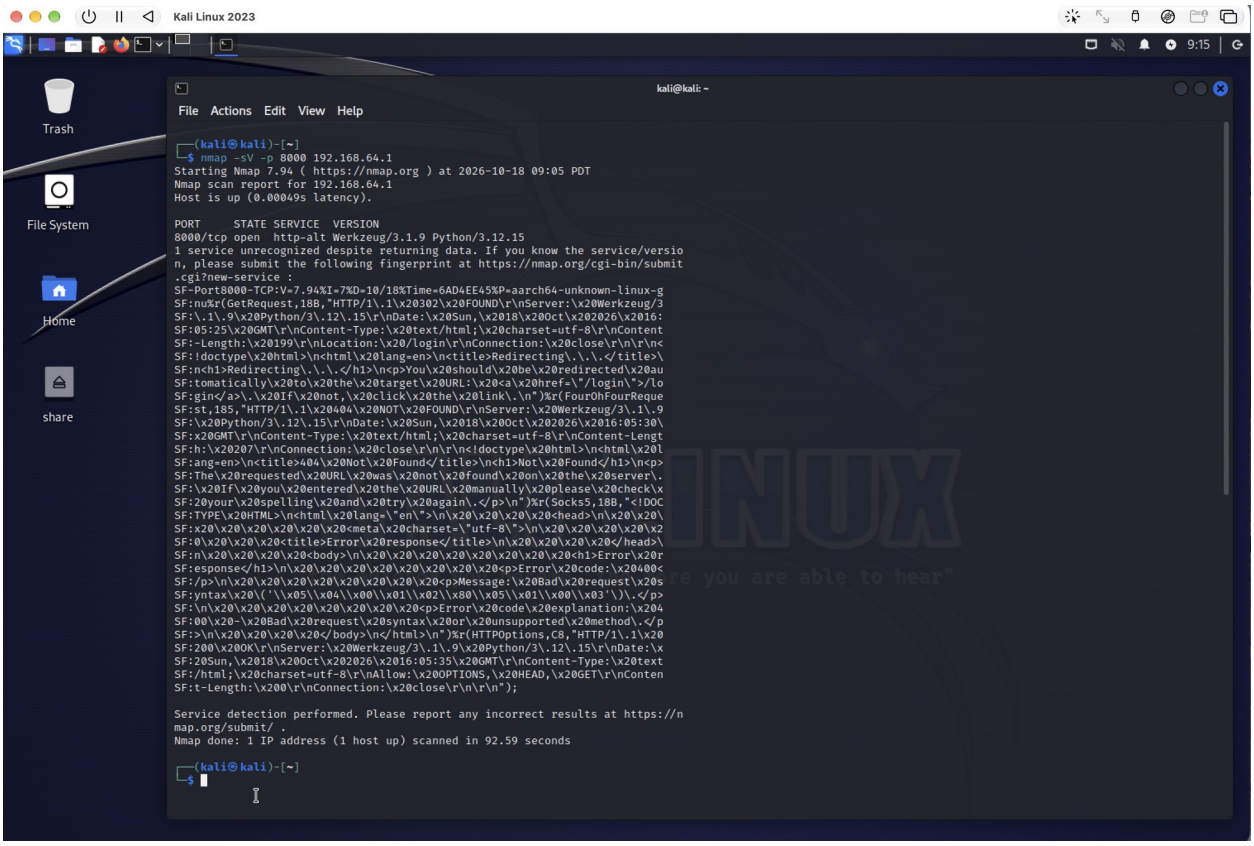
Phase 1: Offensive Engagement

Reconnaissance

The engagement began with reconnaissance, the stage in which an attacker gathers information about a target before attempting to break in. A port and service scan with nmap identified the backend software as Werkzeug and Python, revealing that the site was a Flask application (Figure 2). Directory enumeration with gobuster, which rapidly guesses common page names to discover reachable pages, mapped the attack surface (the set of exposed entry points), finding the login, dashboard, upload, and ping endpoints and generating 4,614 requests for pages that did not exist (Figure 3). In the MITRE ATT&CK framework, a standard catalog that assigns every known attacker technique a name and identifier, this is classified as Active Scanning (T1595; MITRE, 2025).

Figure 2

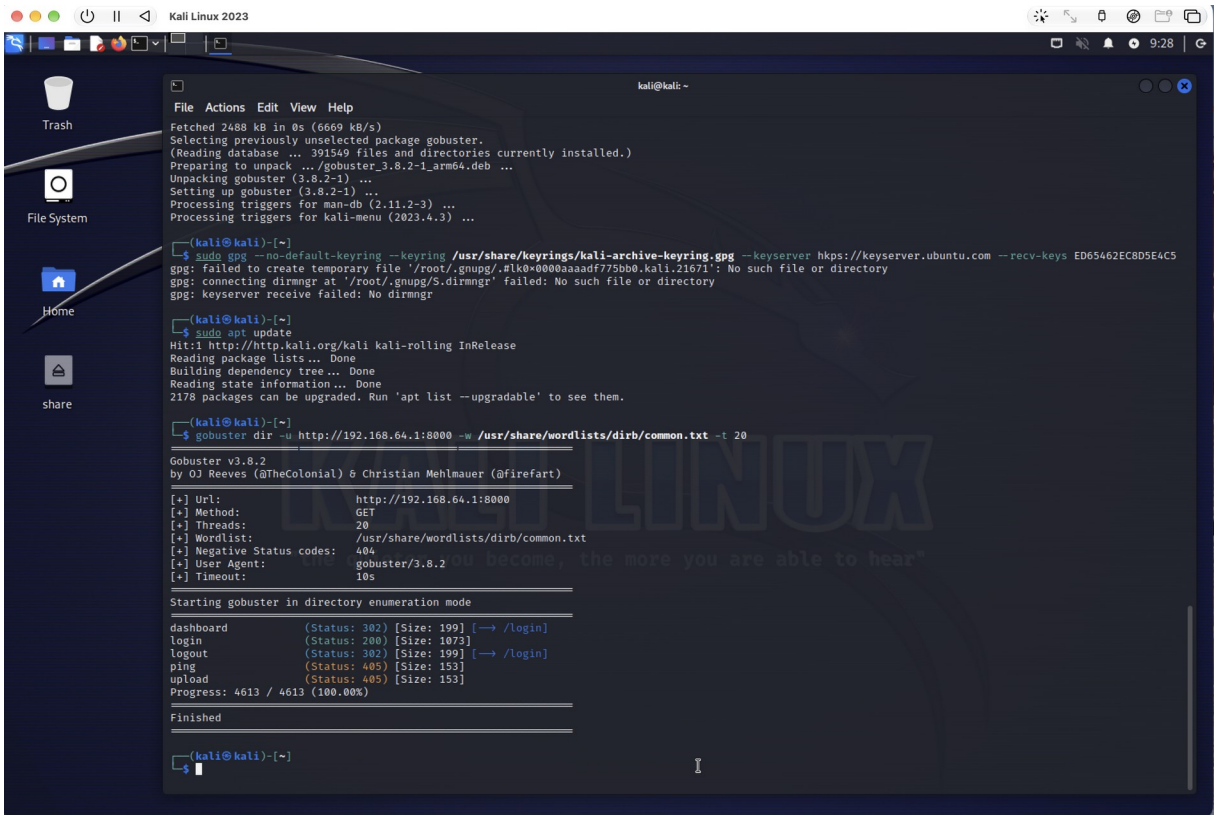
Service Identification with nmap



Note. The scan fingerprints the backend as Werkzeug and Python, identifying the application framework.

Figure 3

Endpoint Enumeration with gobuster



Note. Directory enumeration discovers the login, dashboard, upload, and ping endpoints.

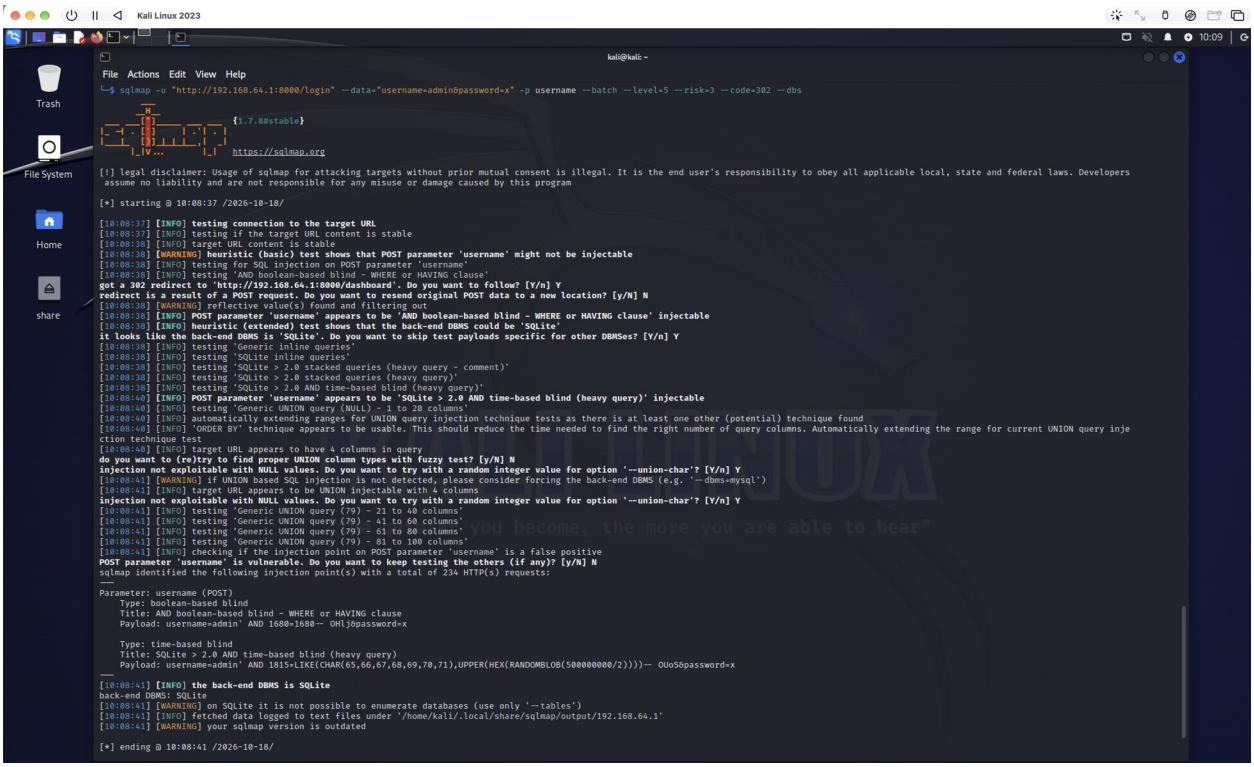
Initial Access

Two independent methods of initial access were demonstrated. The first was SQL injection, a technique in which specially crafted input sent to a website manipulates the database query behind it. The automated tool sqlmap confirmed that the login's username field was injectable and used it to extract the entire user table (Figure 4). Entering the classic payload admin' OR '1'='1 was by itself enough to bypass authentication and sign in as the administrator with no valid password. The second method was credential brute force, the rapid automated guessing of passwords. Because the login enforced no rate limiting or account lockout, the tool hydra tried a list of common passwords and recovered a valid credential, the account ajadi with

the password password123, in about one second (Figure 5). In MITRE ATT&CK terms these are Exploit Public Facing Application (T1190) and Brute Force (T1110).

Figure 4

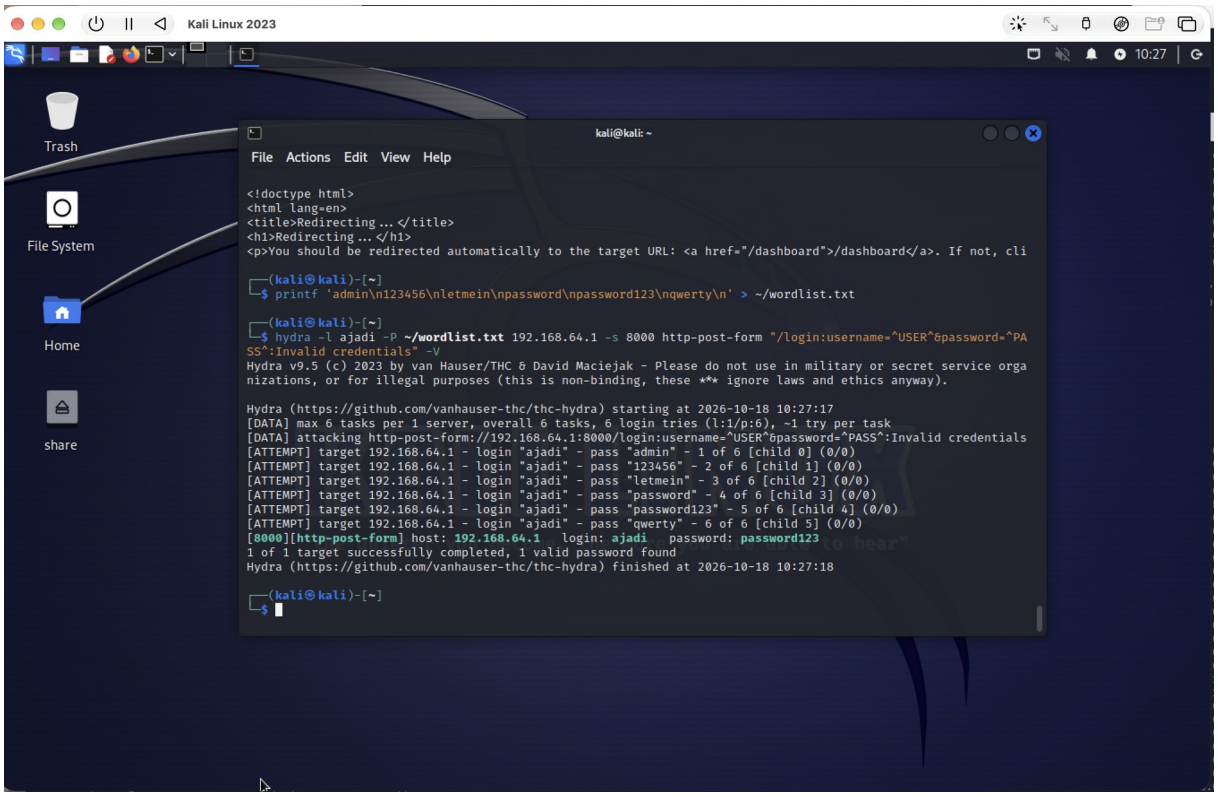
SQL Injection Confirmation with sqlmap



Note. sqlmap confirms the injectable parameter and identifies the SQLite backend.

Figure 5

Credential Brute Force with hydra



Note. With no account lockout, hydra recovers a valid credential almost instantly.

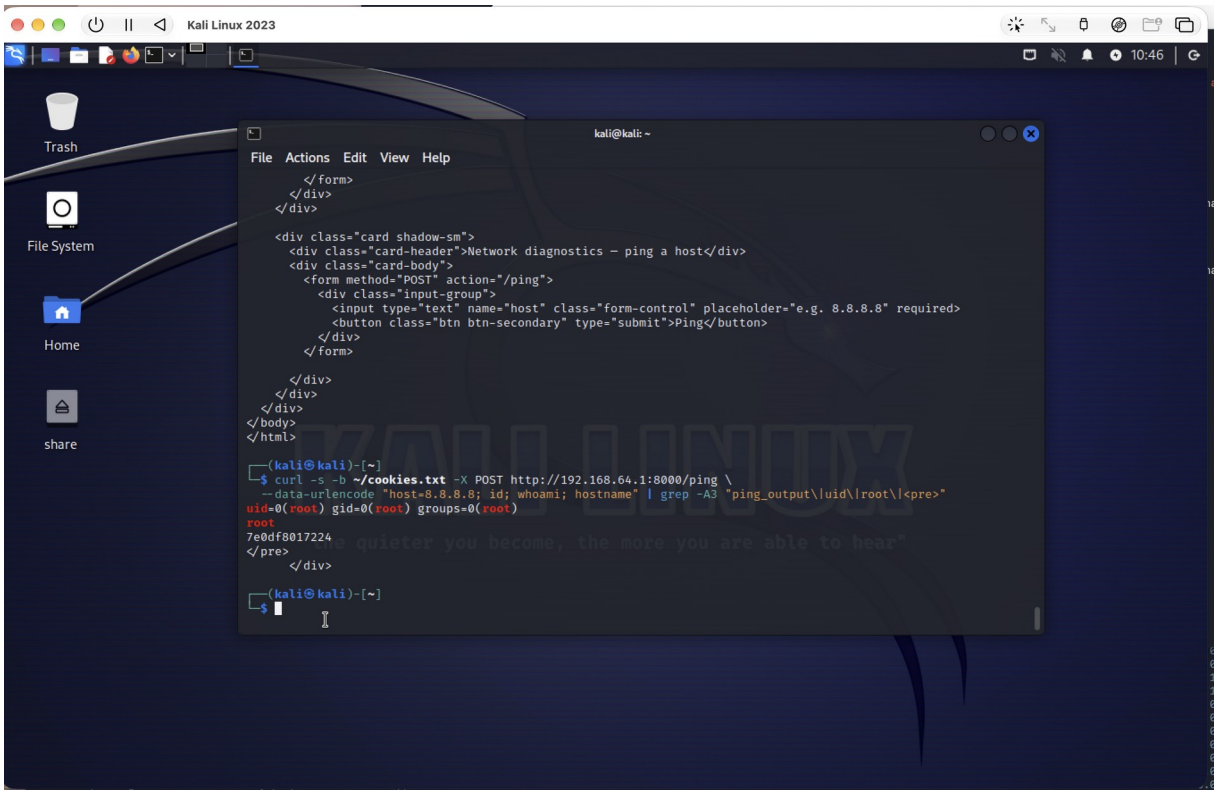
Foothold, Execution, and Exfiltration

Using the recovered credential, the attacker uploaded a script through the unrestricted file upload, planting a web shell, which is a malicious file placed on a server to give the attacker a foothold (classified as Server Software Component: Web Shell, T1505.003). The attacker then exploited the ping tool for operating system command injection: because the tool passed user input directly to a system shell (the program that runs commands), appended commands executed on the server. These ran as the root user, confirmed by the output uid=0(root), giving the attacker remote code execution, the ability to run arbitrary commands on the server (Figure 6; Command and Scripting Interpreter, T1059). Finally, the attacker performed data exfiltration,

copying the entire user database out of the site. Because passwords were stored in plaintext, every credential was exposed immediately (Figure 7; Exfiltration Over C2 Channel, T1041).

Figure 6

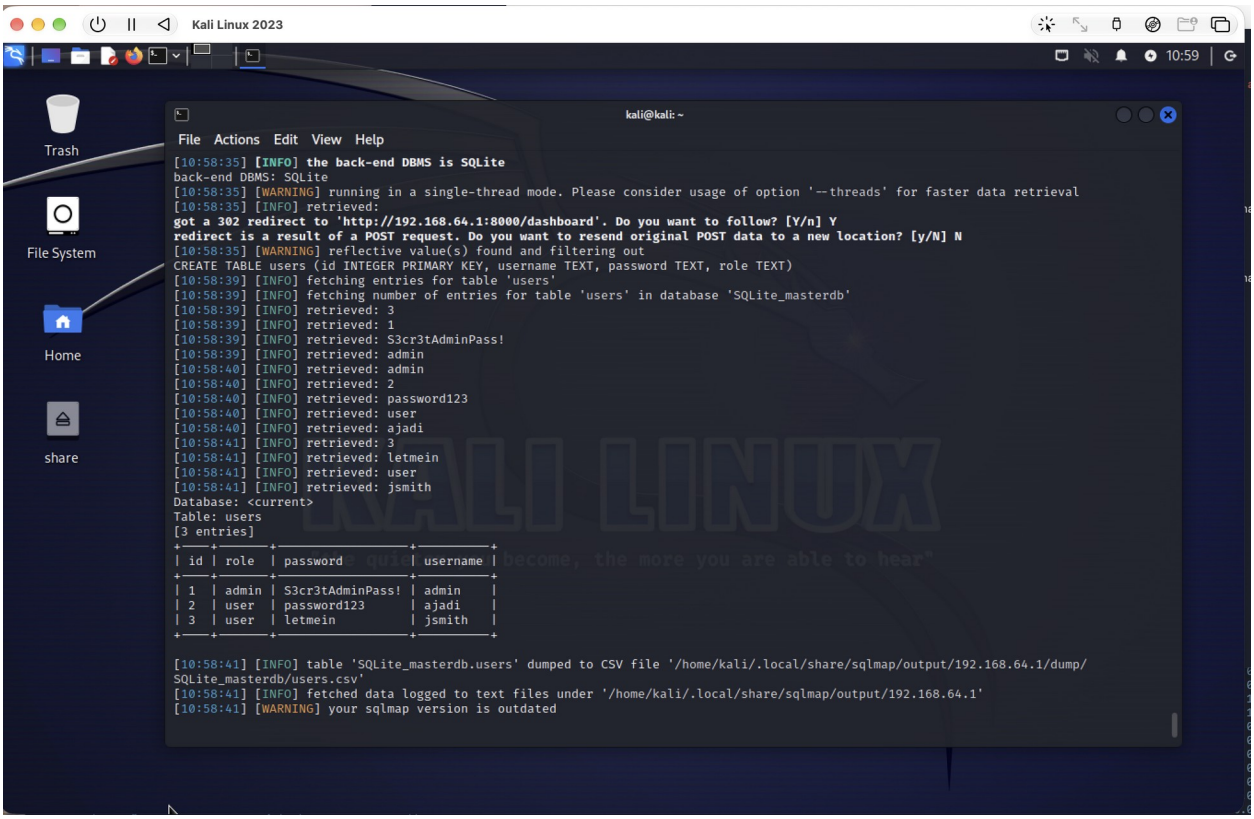
Remote Code Execution as Root via Command Injection



Note. The injected commands return uid=0(root), confirming code execution at the highest privilege level.

Figure 7

Exfiltrated Credential Database



Note. The full user table, with passwords in plaintext, is extracted from the target.

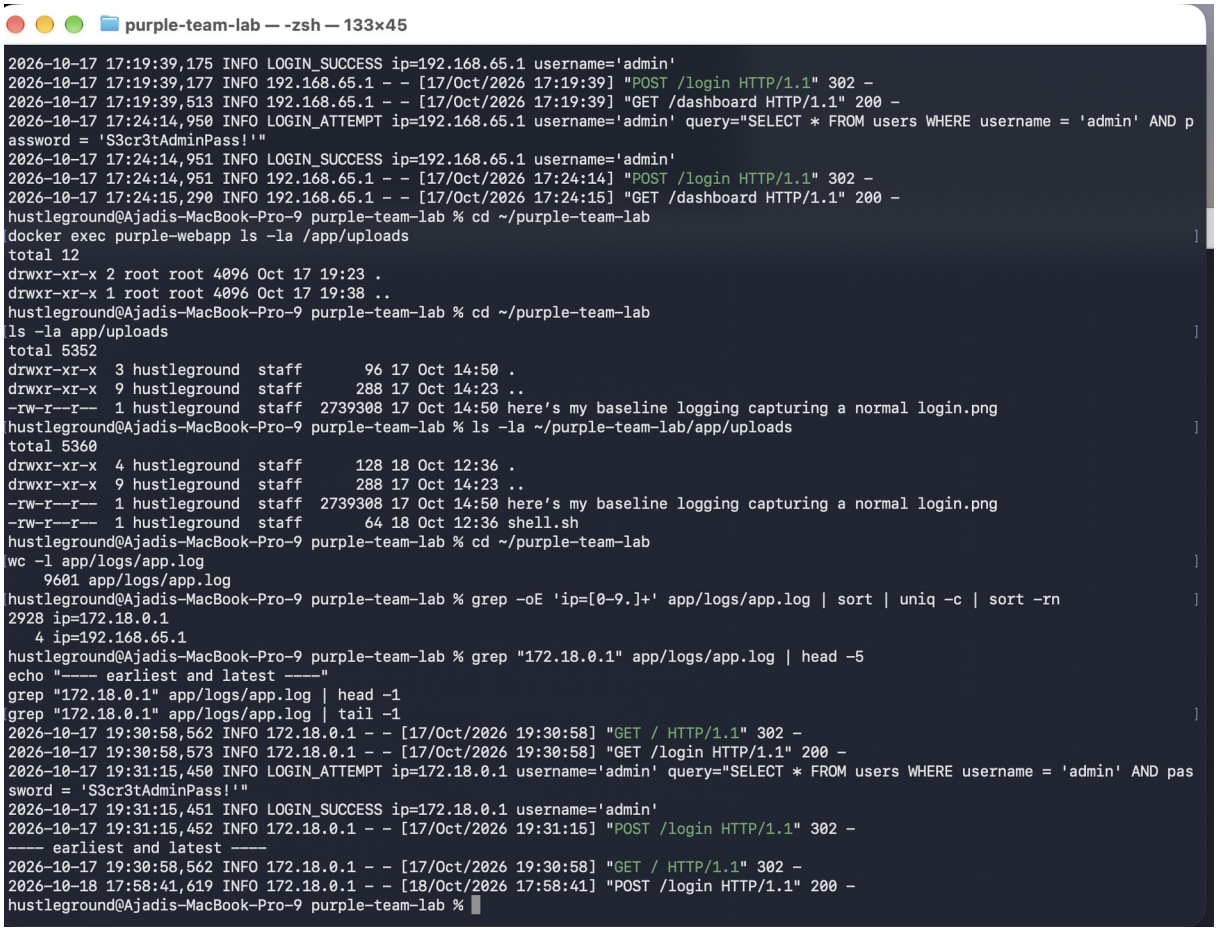
Phase 1: Incident Detection and Forensic Analysis

The author then adopted the role of a SOC analyst with no prior knowledge of the attack, reconstructing the incident using only the website's log. Frequency analysis of source addresses (counting how often each one appears) showed that a single host accounted for more than 99.9% of all logged activity, immediately identifying the attacker (Figure 8). A noteworthy forensic point arose: because the website ran inside a container (a lightweight, isolated environment), the log recorded the attacker as the container's internal gateway address (172.18.0.1) rather than the true client address. This reflects a common real world condition in which an application behind a proxy or load balancer, intermediary equipment that forwards network traffic, records the

intermediary's address unless it is specially configured to preserve the original. All timestamps were in Coordinated Universal Time (UTC), the global reference time.

Figure 8

Source Address Frequency Analysis



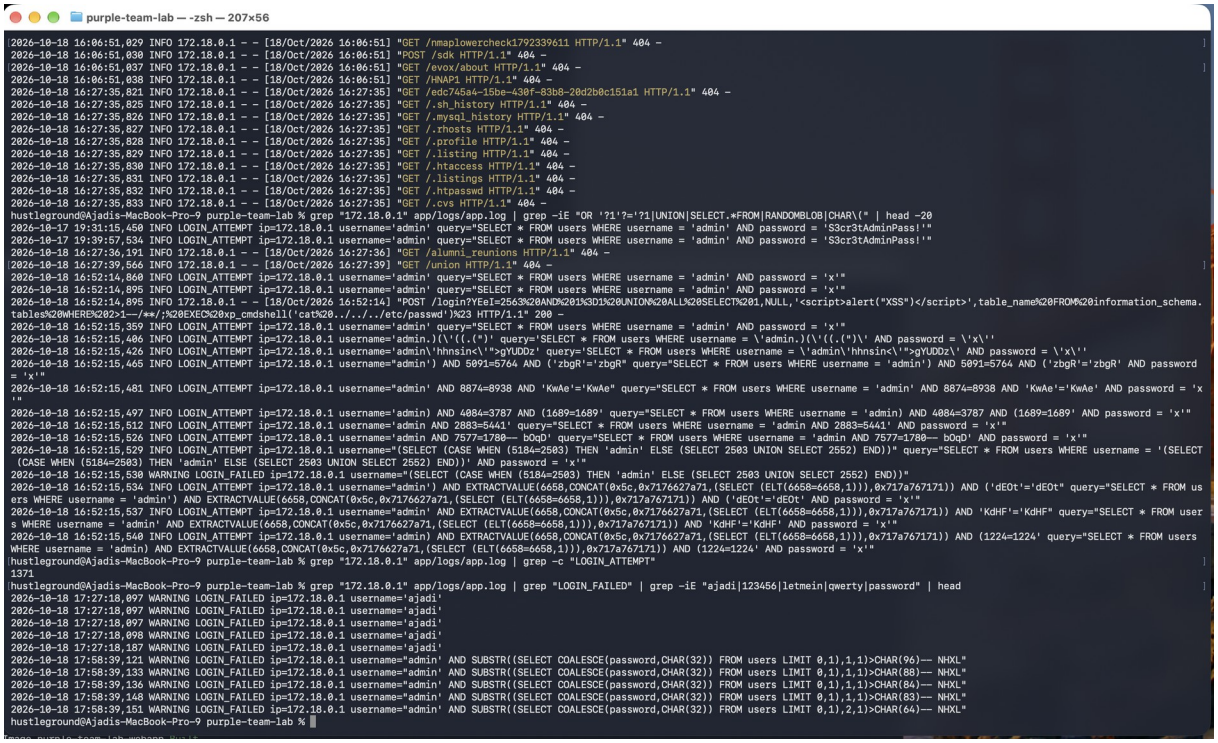
Note. A single source accounts for almost all logged activity, isolating the attacker.

Each stage of the attack left a distinct signature in the log. Reconnaissance produced 4,614 responses with HTTP status 404 (page not found), including nmap's characteristic probe strings and a sweep for sensitive files (Figure 9). The injection attempts were logged verbatim, capturing the malicious payloads in full (Figure 10). The brute force appeared as a rapid cluster

of failed login events from one source (Figure 11). Each behavior was mapped to its MITRE ATT&CK technique and compiled into a timeline and a formal incident report.

Figure 9

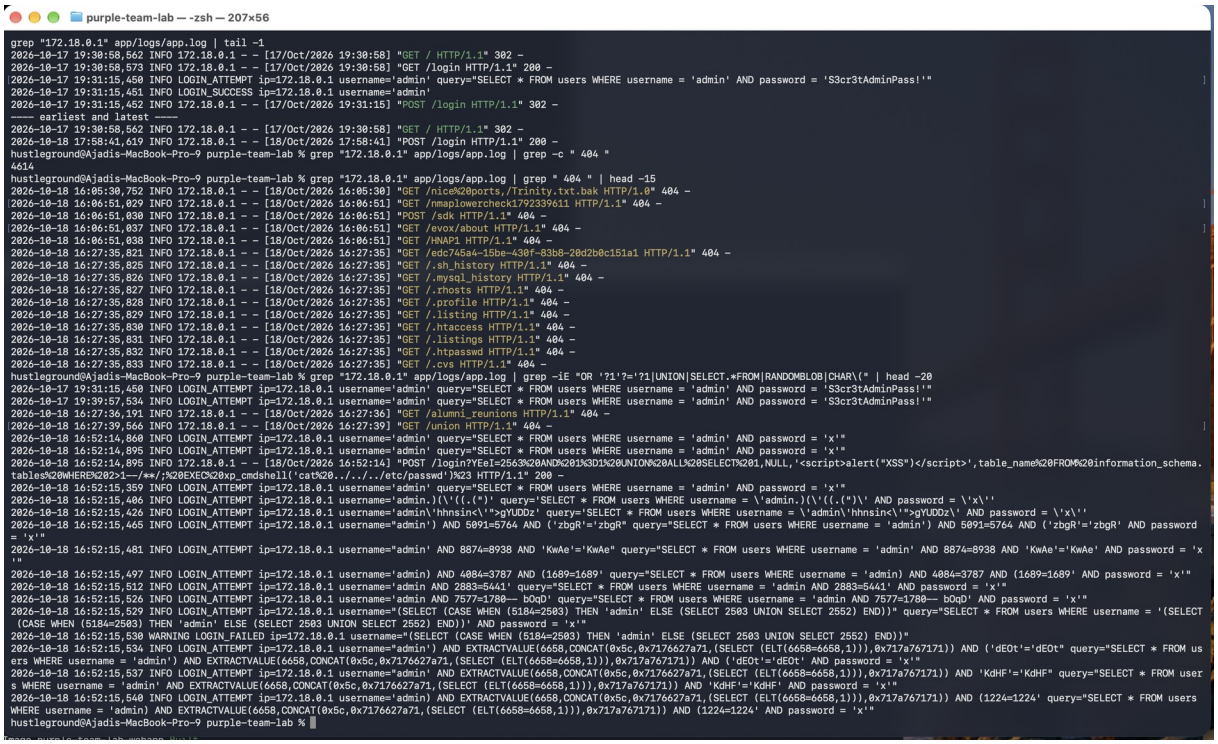
Reconnaissance Signature in the Log



Note. A burst of HTTP 404 responses, including nmap probe strings and a search for sensitive files, evidences automated scanning.

Figure 10

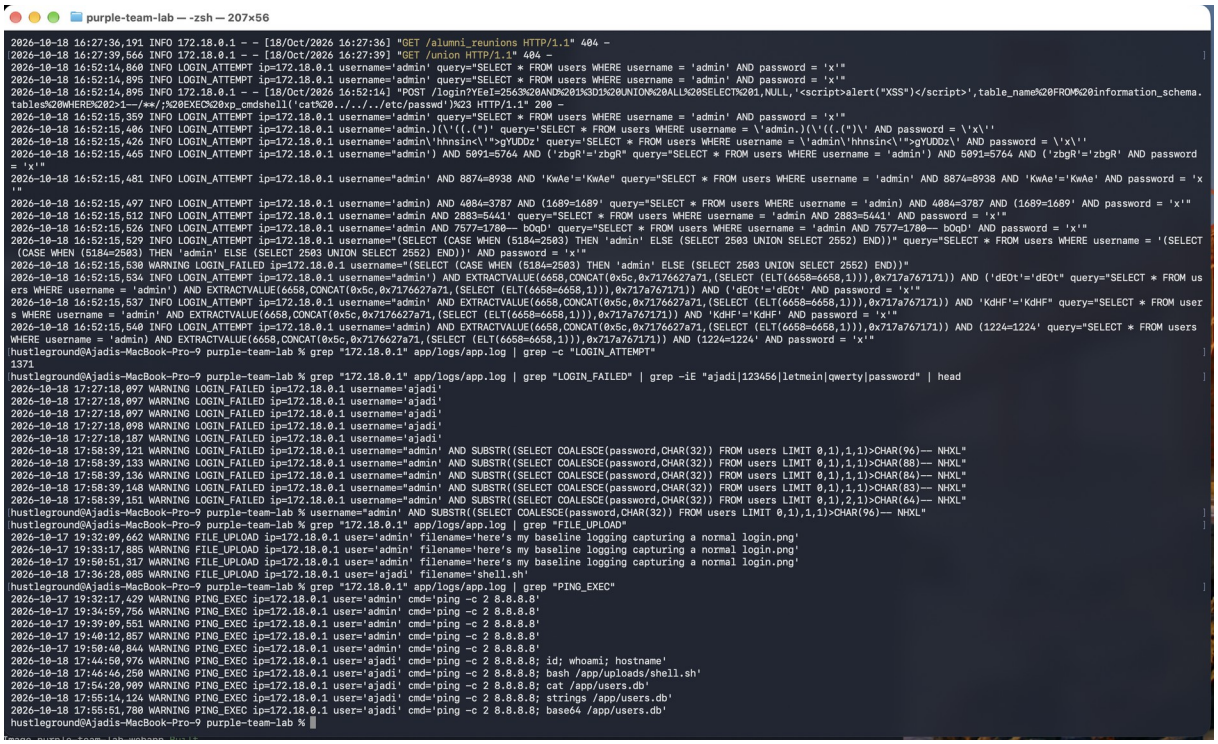
Captured SQL Injection Payloads



Note. The log preserves the attacker's injection payloads verbatim.

Figure 11

Credential Brute Force Signature



Note. Many failed logins from one source within milliseconds indicate automated guessing; the web shell upload and injected commands are also visible.

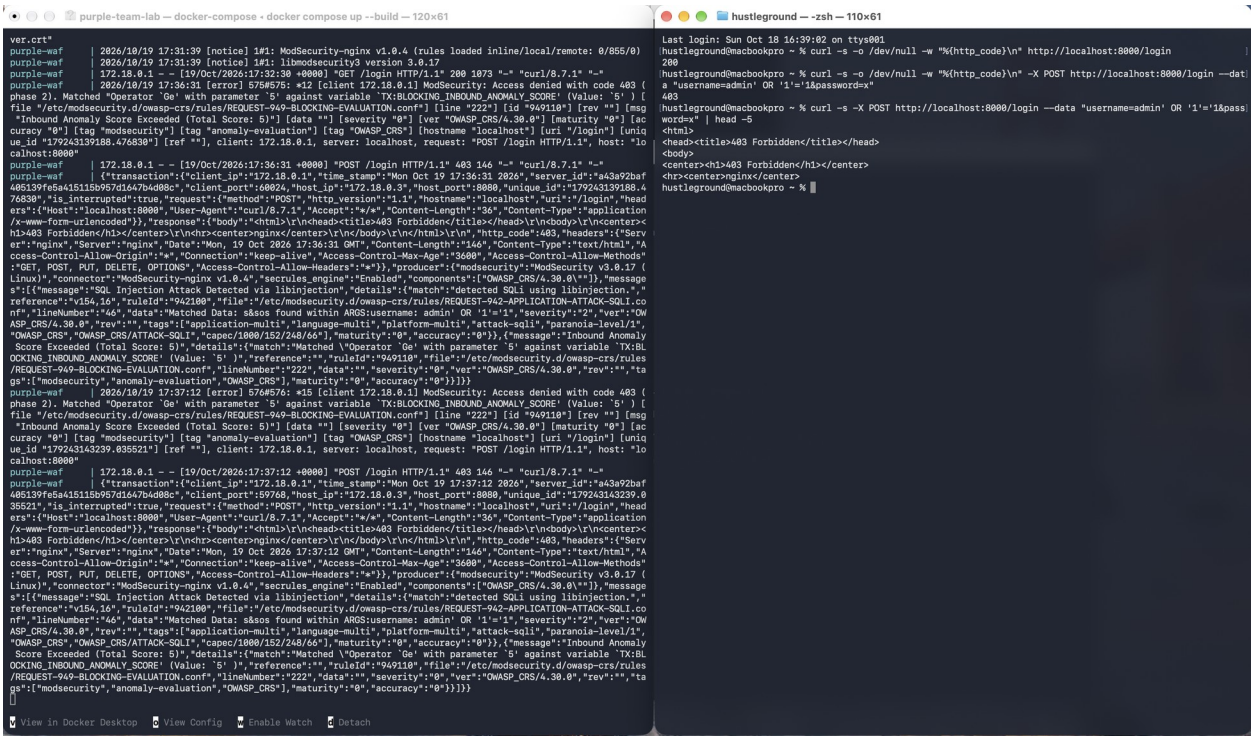
Phase 2: Defensive Implementation and Validation

Web Application Firewall

In the second phase, the WAF was placed in front of the website and the identical attack chain was repeated. The manual SQL injection that had previously bypassed authentication now returned HTTP status 403 (Forbidden, meaning access was denied), blocked by Core Rule Set rule 942100 (Figure 12). When sqlmap was run again with its cache cleared, it reported that the target was protected by a firewall and extracted no data (Figure 13). The command injection that had produced remote code execution was likewise blocked with a 403 response (Figure 14).

Figure 12

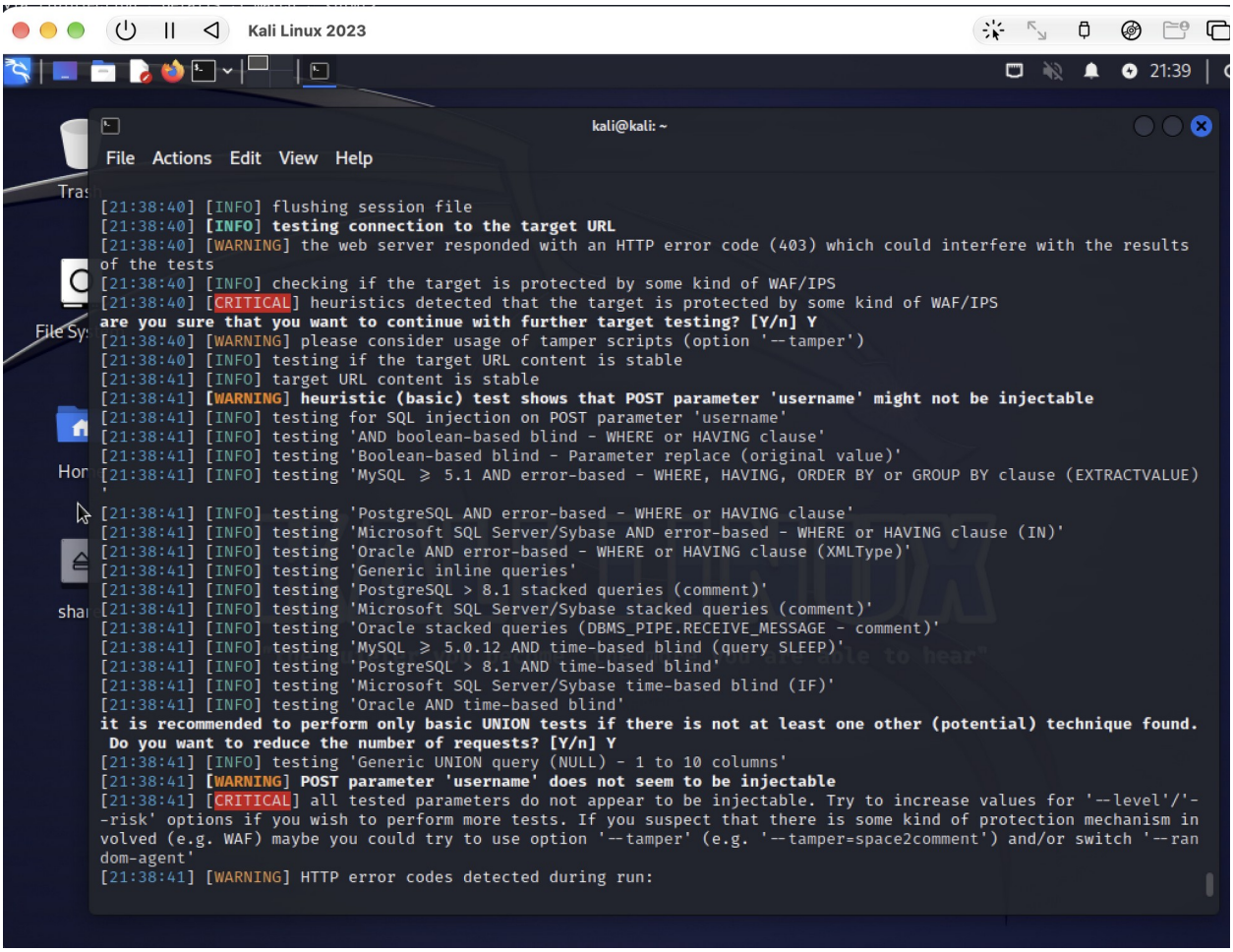
WAF Blocking the SQL Injection



Note. The payload that bypassed authentication in Phase 1 now returns HTTP 403, blocked by Core Rule Set rule 942100.

Figure 13

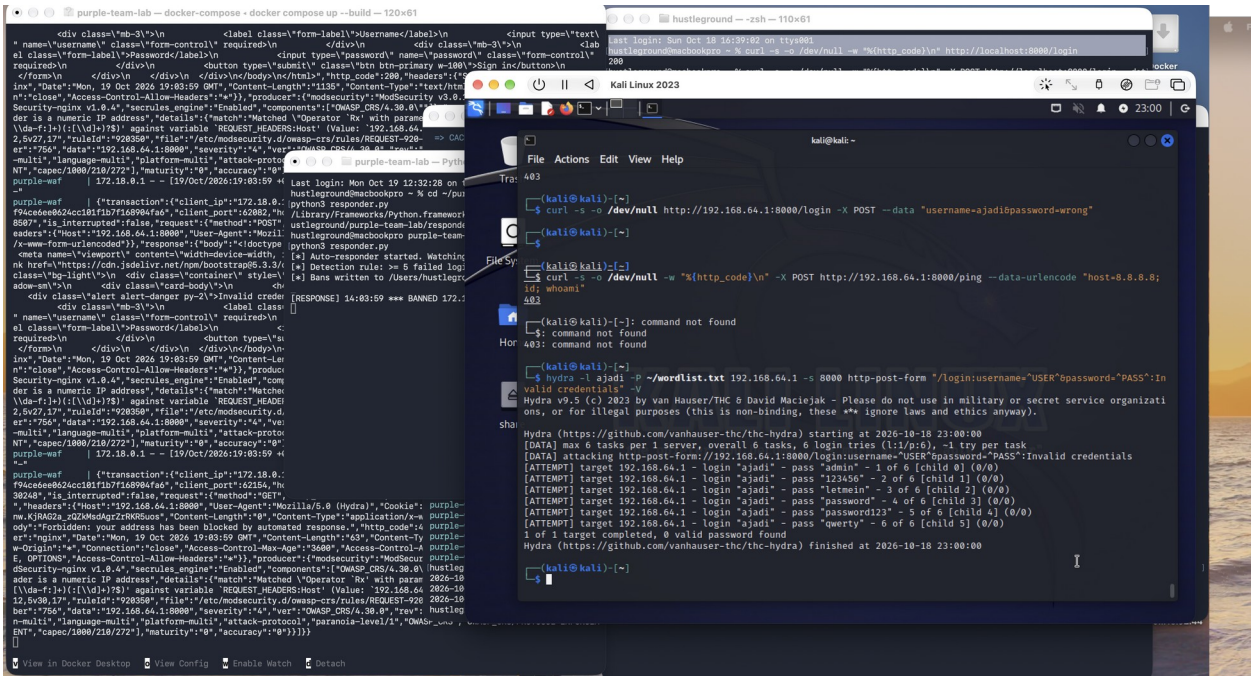
sqlmap Defeated by the WAF



Note. Behind the firewall, sqlmap finds the parameter no longer injectable and extracts no data.

Figure 14

WAF Blocking the Command Injection



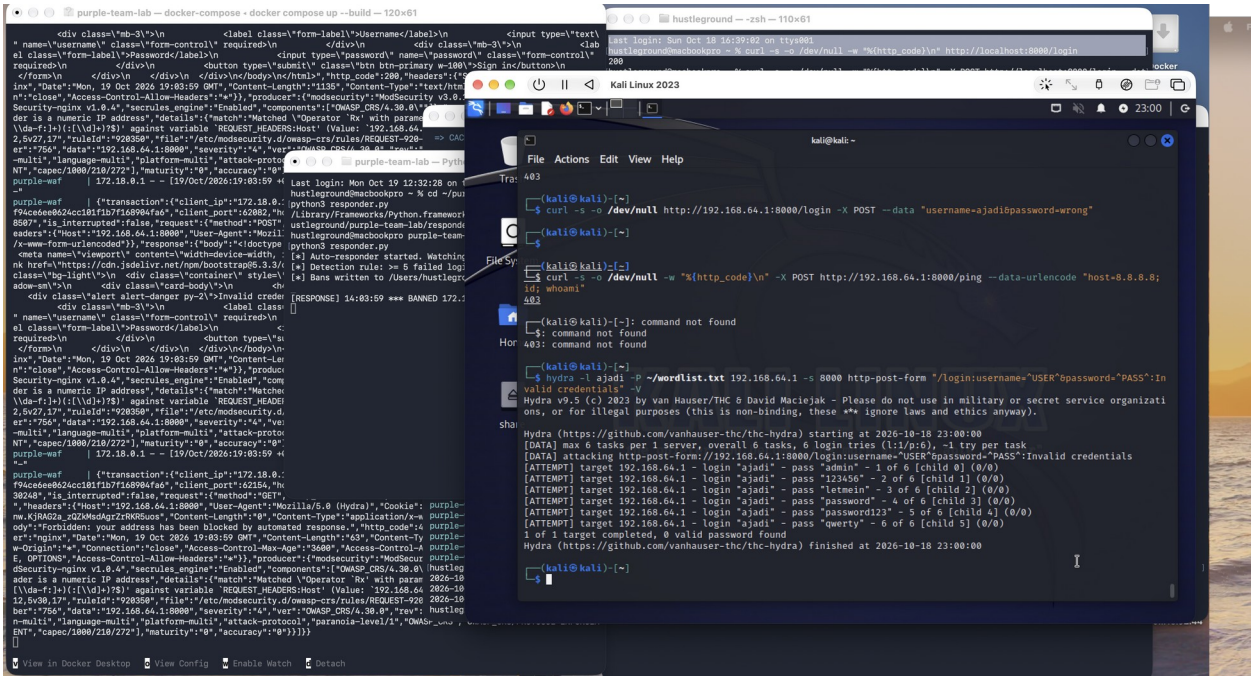
Note. The command injection that produced root access in Phase 1 now returns HTTP 403.

Behavioral Detection and Automated Response

The one attack the WAF could not stop was credential brute force. A single login attempt resembles a legitimate submission and carries no malicious signature, so the firewall has no pattern to match; the attack is only recognizable in aggregate, as a burst of failures. To close this gap, the custom automated response program monitored the log, counted failed login events per source address within a sliding time window (a short rolling span of time), and automatically added any address that crossed a threshold to a blocklist enforced by the application. When the brute force was repeated, the program detected the burst and banned the attacker's address within seconds. Hydra recovered no valid password, and all further requests from the banned address were refused with HTTP 403 (Figure 15). The ban and the resulting blocked requests were confirmed in the log (Figure 16).

Figure 15

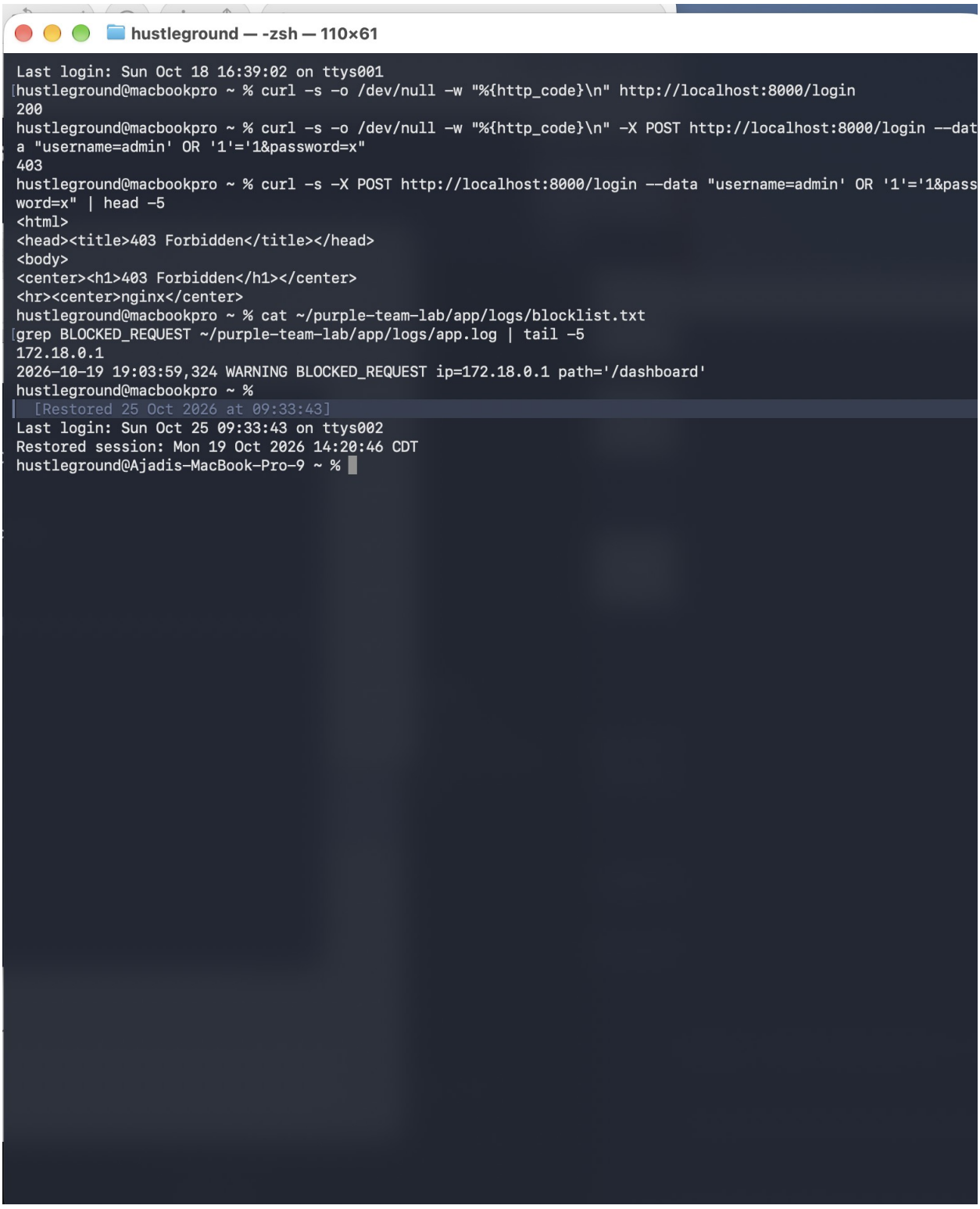
Automated Response Defeating the Brute Force



Note. The program bans the source mid attack; hydra recovers no valid password and the attacker is locked out with HTTP 403.

Figure 16

Enforcement of the Automated Ban



Note. The program records the ban, and the log confirms the attacker's requests being refused.

Results

The two phases produced a direct before and after comparison, summarized in Table 1. Every attack that succeeded against the undefended website in Phase 1 was prevented in Phase 2. The injection attacks were stopped at the network edge by the firewall's signature based detection (recognizing known malicious patterns), while the brute force, invisible to the firewall, was stopped by behavioral detection (recognizing the suspicious burst) and automated response.

Table 1

Attack Outcomes With and Without Defensive Controls

Attack (MITRE ATT&CK)	**Phase 1: Undefended**	**Phase 2: Defended**
SQL injection authentication bypass (T1190)	Administrator access, no password	Blocked, HTTP 403 (CRS 942100)
Automated database dump with sqlmap (T1190)	Full credential database stolen	WAF detected; no data extracted
Command injection, remote code execution (T1059)	Code execution as root	Blocked, HTTP 403 (CRS 932xxx)
Credential brute force (T1110)	Credentials cracked in about 1 second	Detected; source auto banned; none cracked
Subsequent access	Not applicable	Attacker locked out, HTTP 403

Note. CRS = OWASP Core Rule Set. HTTP 403 (Forbidden) means the request was denied.

Discussion

This assessment demonstrates how preventive controls and detective controls complement one another. The WAF was highly effective against attacks with recognizable malicious signatures, such as the SQL injection and command injection, blocking them before they reached the application. It was ineffective against credential brute force, which is malicious only in aggregate and looks normal on a per request basis. That gap motivated the behavioral automated response component. The result reinforces the principle of defense in depth: no single

control is sufficient, and effective security comes from layering controls that each address a different class of threat (Scarfone et al., 2008).

Several limitations should be noted. The lab used a single attacking host, so banning one address fully neutralized the brute force; a distributed attack from many addresses would require additional controls such as rate limiting. The vulnerabilities were introduced deliberately and were individually straightforward, whereas real applications often hide more subtle flaws. Finally, the WAF ran at a conservative paranoia level (a setting that trades sensitivity for fewer false alarms); a more aggressive setting would detect more attacks but require additional tuning.

Conclusion

This purple team assessment traced a website from full compromise to effective defense. In the undefended phase, a realistic attack chain achieved administrator access without a password, remote code execution as root, and complete exfiltration of the credential database, all of which were reconstructed from log evidence and mapped to the MITRE ATT&CK framework. In the defended phase, a web application firewall and a custom automated response program prevented every attack that had previously succeeded. The project provides concrete, evidence backed demonstration of both the offensive techniques and the layered controls that counter them, and it underscores the value of combining signature based prevention with behavioral detection and automated response.

References

MITRE. (2025). MITRE ATT&CK®. https://attack.mitre.org/

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

OWASP Foundation. (2024). OWASP Core Rule Set. https://coreruleset.org/

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). Technical guide to information security testing and assessment (NIST Special Publication 800-115). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-115

Stuttard, D., & Pinto, M. (2011). The web application hacker's handbook: Finding and exploiting security flaws (2nd ed.). Wiley.